



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

LIBS: Libraries Identification in Statically Linked Binaries using Word Embeddings

Tesi di Laurea Magistrale in
Computer Science and Engineering - Ingegneria Informatica

Author: **Matteo Rizzi**

Student ID: 969247

Advisor: Prof. Mario Polino

Co-advisors: Lorenzo Binosi, Stefano Zanero

Academic Year: 2022-2023

Abstract

In binary code analysis, identifying which libraries have been compiled within a statically linked binary can be of great interest for some scenarios, such as bug searching or reverse engineering. However, this identification can be challenging due to the diversity of compilers, compilation settings (such as optimization levels) and software architectures, which introduce differences in the compiled code. Moreover, libraries can have many different versions, making their identification even harder. In this thesis we propose an architecture-independent approach, implemented into a tool called LIBS (Libraries Identification in Binaries), to identify which libraries have been compiled inside a statically linked binary and to recognize their correct versions. Our approach works by performing library functions identification between a statically linked binary and one or more static libraries to find the similarities between their functions. It is based on a word embedding model to extract a semantic feature from each function; this feature is used, together with other structural and syntactical ones, to measure the similarity between two functions. To be able to work with different software architectures, our approach works on top of an intermediate representation of the code, the LLVM IR. Experimental validation, performed for two different software architectures, MIPSel and x86_64, shows that our approach can achieve good results in identifying which libraries have been compiled inside a statically linked binary, with a Precision of 87% for MIPSel and 93% for x86_64. Moreover, experimental results show that LIBS is able to identify the libraries even between different versions and to correctly provide their exact version with an Accuracy of 72% for MIPSel and 84% for x86_64.

Keywords: Binary Code Analysis, Library Functions Identification, Reverse Engineering, Word Embedding, LLVM IR

Sommario

Durante l'analisi di file binari compilati staticamente, identificare quali librerie sono presenti al loro interno può essere di grande interesse per ambiti quali la ricerca di bug o il reverse engineering. Tuttavia, questa identificazione può risultare difficile a causa di differenti compilatori, impostazioni di compilazione (come i livelli di ottimizzazione) e architetture software, che introducono differenze nel codice compilato. Inoltre, le librerie possono avere molte versioni diverse, il che rende ancora più difficile la loro identificazione. In questa tesi proponiamo un approccio multi-architettura, implementato in un tool chiamato LIBS (Libraries Identification in Binaries), per identificare quali librerie sono presenti all'interno di un binario compilato staticamente e riconoscere la loro esatta versione. Il nostro approccio compara un binario compilato staticamente e una o più librerie statiche per trovare similarità tra le loro funzioni. Si basa su un modello di word embedding per estrarre la semantica di ogni funzione per misurare, insieme ad altre proprietà di tipo strutturale e sintattico, la similarità tra due funzioni. Per essere in grado di lavorare con differenti architetture software, il nostro approccio lavora su una rappresentazione intermedia del codice, la LLVM IR. Risultati sperimentali derivanti da test condotti su due diverse architetture software, MIPSel e x86_64, mostrano che il nostro approccio è in grado di ottenere buoni risultati nell'identificare quali librerie sono presenti all'interno di un binario compilato staticamente, con una Precisione dell'87% per MIPSel e del 93% per x86_64. Inoltre, i risultati sperimentali dimostrano che LIBS è in grado di identificare le librerie anche tra versioni differenti e di fornire correttamente la loro esatta versione con una Accuratezza del 72% per MIPSel e dell'84% per x86_64.

Parole chiave: Binary Code Analysis, Library Functions Identification, Reverse Engineering, Word Embedding, LLVM IR

Contents

Abstract	i
Sommario	iii
Contents	v
1 Introduction	1
2 Motivation	3
2.1 Problem Statement	4
2.2 State of the Art	4
2.3 Goals and Challenges	5
3 Approach	7
3.1 Approach Details	7
3.1.1 LLVM IR Parsing	8
3.1.2 Feature Vectors Generation	9
3.1.3 Functions Matching	10
4 Implementation	11
4.1 <i>IR Loader</i>	11
4.2 <i>Embeddings Manager</i>	12
4.3 <i>Functions Matcher</i>	13
4.4 <i>Results Aggregator</i>	13
5 Experimental Validation	17
5.1 Dataset	18
5.2 Setup	19
5.3 Parameters Validation	20
5.4 Experiment 1	21

5.4.1	Experiment 1.1	22
5.4.2	Experiment 1.2	23
5.4.3	Experiment 1.3	23
5.5	Experiment 2	24
5.5.1	Experiment 2.1	24
5.5.2	Experiment 2.2	26
5.5.3	Experiment 2.3	27
5.6	Use Cases Flexibility	29
6	Limitations and Future Works	31
7	Conclusions	33
	Bibliography	35
	List of Figures	37
	List of Tables	39
	Ringraziamenti	41

1 | Introduction

Binary code analysis is essential when dealing with compiled binary files whose source code is not available, such as malware or commercial off-the-shelf software. It consists in reconstructing useful information from the compiled code, normally by using tools such as IDA Pro [1] to first decompile the machine code into a more readable, higher-level code and manually analyzing it for reverse engineering purposes, bug searching or to detect potential malicious behaviors.

When dealing with statically linked binaries, i.e., those that contain also the code of the libraries they use, an analyst may find himself analyzing a portion of decompiled code that is actually part of an external library; it is in fact hard to distinguish the actual code of the binary from the code that is imported by including third-party libraries. Thus, it would be beneficial for the analyst to be able to know whether the decompiled code he is analyzing comes from a third-party library and, if so, which one; this information would in fact allow the analyst to skip that portion of code, significantly reducing the time and effort needed to analyze the binary, since he can directly analyze the original source code of the library. Moreover, knowing that the code of a specific library is inside a statically linked binary may provide useful information about possible vulnerabilities that could be found during the analysis process: in fact, if the identified library has a publicly-disclosed vulnerability, then this vulnerability may also be present inside the binary itself. It thus becomes clear how important it is to recognize which portions of decompiled code from a statically linked binary come from third-party libraries. This is called library functions identification.

Few library functions identification tools and methods exist. The already mentioned IDA Pro decompiler contains an utility called FLIRT [2] that can identify functions that come from known standard libraries. Other approaches make use of structural features of the code, such as the one proposed in [3], to recognize library functions. However, these approaches mostly focus on only one type of feature of the code, capturing only some aspects of it but missing some others, such as its semantics.

Code semantics is important when taking into consideration how the code is compiled.

In fact, different compiler versions or compiler settings, such as the optimization levels, introduce variations in the final compiled code; because of this, the structure or syntax of a function may change, but not its semantics, which remains unaltered. It is thus necessary to develop a library functions identification approach that can be robust to code variations, which can be achieved by simultaneously considering structural, syntactical and semantic features of the functions to identify their similarities.

In this thesis we propose an approach that can be used to:

- Identify which libraries have been compiled within a statically linked binary.
- Recognize the correct version of the identified libraries.
- Provide a mapping between the binary's functions and the libraries' ones.

The approach we propose performs library functions identification to match pairs of functions between a binary and a library by extracting structural, syntactical and semantic features from them. To capture the semantics of the code it makes use of a FastText [4] word embedding model that extracts a word embedding from each function. To support many software architectures, it works upon an intermediate representation of code, the LLVM IR.

We implement our approach in a tool called LIBS (which stands for Libraries Identification in Binaries) and test it for two different architectures, MIPSel and x86_64. Experimental results show that our approach is able to:

- Correctly identify the majority of compiled libraries within a statically linked binary.
- Correctly recognize the exact version for the majority of identified libraries.
- Correctly provide a mapping between the binary functions and the library ones.

In summary, our contributions over the state of the art are the following:

- We propose an approach based on a FastText word embedding model and on the LLVM IR to identify which libraries have been compiled inside a statically linked binary.
- We implement our approach in a tool called LIBS and we determine its optimal configuration of parameters.
- We evaluate our tool on a dataset composed by 600 statically linked binaries and 750 static libraries for two different software architectures, MIPSel and x86_64.
- We demonstrate that our approach can correctly identify the majority of compiled libraries within a statically linked binary and that it can recognize their correct version.

2 | Motivation

Binary analysis stems from the lack of freely available source code for a particular compiled binary; this can be the case of malware or commercial off-the-shelf software. To reverse-engineer these compiled binaries or to detect bugs or potential malicious code inside them an analyst has to first decompile them using a specialized software (such as IDA Pro) to obtain an higher-level representation that tries to reconstruct the functions of the original code; he then needs to manually analyze those functions to extract useful information. The difficulty and time required by this analysis process are increased by the practice of stripping, which removes from the compiled binary all debug symbols, all variable names and all function names and boundaries.

In software development it is common to use open source third-party libraries. When compiling code that uses external libraries there are two possibilities: one is to dynamically link the external libraries to the binary, which means that the libraries are imported at runtime and so their code is not present inside the compiled binary; the other is to statically link the external libraries, which means that the libraries code is included in the compiled binary. To be statically linked the libraries must be compiled as static libraries; these are a collection of compiled files, called compilation units, archived in a single *.a* file. It is important to note that not all the compilation units of a static library are by default imported into the binary, but only the ones containing code that is actually needed for the binary to correctly run.

As a consequence, for statically linked binaries the analysis process becomes longer because there is much more code to inspect, since they contain code that comes from external libraries and that is not easily distinguishable from the actual binary's code. In this situation an analyst will find himself analyzing some decompiled functions that come from external libraries. Thus, being able to automatically identify which functions from a statically linked binary come from external libraries and, by extension, which libraries have been compiled inside the binary, brings benefits to the analyst. He could in fact skip the analysis of these decompiled functions to directly inspect their source code, significantly reducing the time and effort needed for the analysis process. Moreover, knowing which libraries have been compiled inside a statically linked binary can give useful hints about

possible vulnerabilities that can be found during the analysis process; in fact, if one of the identified libraries contains a publicly disclosed vulnerability, that same vulnerability may be also present in the binary itself.

2.1. Problem Statement

The problem of identifying which functions of a binary come from external libraries is called library functions identification. We extend the library functions identification problem to the more general one of identifying which libraries have been compiled inside a statically linked binary. This general problem can be divided into the following sub-problems:

- Identifying which libraries have been compiled within a statically linked binary, independently from their versions.
- Recognizing the correct version numbers of the identified libraries.

2.2. State of the Art

Few library functions identification approaches exist. The most known tool is FLIRT, which stands for Fast Library Identification and Recognition Technology; it is an utility embedded into IDA Pro with the purpose of identifying if a function belongs to standard C/C++ libraries. It uses pattern recognition based on the first 32 bytes of the function to try to match it against a database of known standard library functions. This approach has demonstrated to achieve good results, but is limited to the database of known libraries. Another interesting approach is the one proposed in [3]. This is a graph-based approach that performs library functions identification using the Execution Dependence Graph (EDG), an extension of the Control Flow Graph (CFG) of a function; the EDG provides a reliable representation of the data and control dependencies of a function and is used to identify similar functions via graph isomorphism.

However, none of the above methods take into account the semantics of the code, focusing only on its structure and syntax. Moreover, no tools exist in the state of the art to solve the two sub-problems stated in Section 2.1. So we have expanded our analysis of the state of the art to also include some tools that perform binary code similarity detection, which is a generalization of the library functions identification problem. Binary code similarity detection techniques can be generally divided into two categories: graph-based approaches and embedding-based approaches.

Graph-based approaches try to model each binary as a graph and use graph isomorphism

techniques to compute the similarity between two binaries; however, graph isomorphism techniques are generally slow and difficult to scale, so they are not very efficient. In this category falls BinDiff [5], the de-facto standard tool for binary diffing. It uses a combination of the Control Flow Graph and the CallGraph (CG) to perform graph isomorphism and find the similarities between two binaries. Some variations of this tool have been proposed, trying to improve its accuracy. Among them, BinSlayer [6] introduces the use of the Hungarian algorithm for bipartite graph matching. A more refined approach is the one of Diaphora [7], an open source binary diffing tool in the form of an IDA Pro plugin; it uses many different techniques to obtain the similarities between two binaries, borrowing some concepts from previous works such as BinDiff but also adding many heuristics to compute the similarity score.

Embedding-based approaches instead try to extract from each binary a set of embeddings, which are numerical vectors of heterogeneous features representing the structural, syntactical and semantic properties of the binary. The similarity between two binaries is given by a distance metric applied to these embeddings. The features composing the embeddings can be extracted manually, using expert knowledge, or automatically using machine learning and clustering techniques. Recently more tools have started to use machine learning to extract embeddings from the code. Among them, DeepBinDiff [8] is an open source tool that uses an unsupervised learning technique to learn how to generate embeddings to model single basic blocks, containing both code semantics and information from the Inter-procedural Control Flow Graph (ICFG); these embeddings are then used to find the most similar basic blocks using a k-hop greedy algorithm. Gemini [9] is a cross-architecture tool for binary diffing, employing a Deep Neural Network to extract embeddings from the Attributed Control Flow Graph (ACFG). Another interesting work is α Diff [10], a cross-architecture tool using machine learning to extract three semantic features from each function of the binary: the first is an intra-function feature and is extracted by a Convolutional Neural Network directly from raw bytes; the other two are an inter-function feature and an inter-module feature, extracted from the CallGraph. These embedding-based approaches have demonstrated to achieve good results in binary diffing, showing the usefulness of machine learning techniques for binary code similarity detection.

2.3. Goals and Challenges

The objective of this thesis is to develop an approach to automatically identify which libraries have been compiled within a statically linked binary and to recognize their correct version. However, to reach this objective we need to face some obstacles. The first problem that emerges is caused by the compilation process. Different compilers and compiler

versions exist, each one producing different compiled code; moreover, even using the same compiler with different settings, such as optimization levels, we obtain some variations in the compiled code. So, the same function compiled with two different optimization levels may exhibit structural or syntactical variations. The second problem arises from the diversity of software architectures. The code of two binaries compiled for two different software architectures, e.g. MIPSel and x86_64, can be very different; this depends on the instruction set and number of bits of the architecture. The third problem is caused by the practice of stripping, which removes all debug symbols and other important information such as variable names and function names and boundaries from the final binary, making the analysis process harder. Finally, it is important to mention the existence of obfuscation techniques, such as packing and encryption, which are often used in malware and sometimes in commercial off-the-shelf software to make the reverse engineering process harder.

Given the above challenges, we set the following goals for our approach:

- Being able to correctly identify which libraries have been compiled within a statically linked binary.
- Being able to correctly recognize the exact version of the identified libraries.
- Being able to correctly provide a mapping between binary functions and library ones.
- Being architecture-independent.
- Being robust against different compiler versions and optimization levels.
- Being independent from variables and functions names.

The goal of being architecture-independent implies that our approach has to be able to work on top of an intermediate representation of the compiled code, while the goal of being independent from variables and functions names means that our approach has to assume to always work with stripped binaries. Lastly, it is important to note that being able to unpack or decrypt binaries is not part of our goals, so our approach assumes to always work with unpacked and decrypted binaries; this means that, in presence of obfuscation techniques, the binaries must first be unpacked or decrypted to be used in our approach.

3 | Approach

To reach the prefixed goals we propose an approach that compares a statically linked binary (target binary) with a set of libraries (query libraries) to identify which are compiled inside the target binary. Each query library is independently compared to the target binary; to find if a query library has been compiled inside the target binary we perform library functions identification on each of its compilation units, one at a time.

The library functions identification approach we propose consists in performing functions matching between the target binary and each compilation unit of the query library. This functions matching has the purpose of constructing, for each compilation unit, a mapping between its functions and the ones of the target binary. A functions mapping is a set $\mathcal{M} = \{P_1, P_2, \dots, P_n\}$ composed of functions pairs $P = (f_{BIN}, f_{CU})$ where f_{BIN} represents a function of the target binary, f_{CU} a function of one compilation unit of the query library and n is the number of functions of the compilation unit. To each possible pair P we assign a similarity score s computed from structural, syntactical and semantic features extracted from the pair itself. This similarity score tells how similar the two functions of the pair are; the higher its value, the greater the probability that the two functions represent a correct match. Between the target binary and a compilation unit there are many possible mappings $\mathcal{M}$; we identify the best mapping as the one with the maximum total similarity score $S = \sum_{i=1}^n s_i$, so the one that contains the pairs that are most likely to be correct matches. Finally, from these mappings we compute an aggregate score for the whole query library to determine if it is compiled inside the target binary.

3.1. Approach Details

The library functions identification approach we propose works at the functions level. So, we need to model the target binary and the query library as a set of functions to then compose many possible functions pairs $P = (f_{BIN}, f_{CU})$ and extract from them their structural, syntactical and semantic features. To model both the target binary and the query library independently from their software architecture we use an intermediate representation of the code; for this purpose we choose to work on top of the LLVM IR, which

allows our approach to be architecture-independent.

Our approach needs the LLVM IR representations of the target binary and of each compilation unit of the query library to perform its functions matching. It parses these LLVM IR files to map them into a data structure to easily retrieve all the information it needs to extract the features. To be robust against the code variations introduced by different compilers versions or optimization levels, our approach also considers the semantics of the code; this is done through a word embedding model that extracts an embedding from each function.

The steps that compose our approach are the following:

1. Load the trained word embedding model.
2. Load the LLVM IR file of the target binary, parse it and extract an embedding from each of its functions.
3. Load the LLVM IR files of all the compilation units of the query library, parse them and extract an embedding from each of their functions.
4. For each compilation unit:
 - 4.1. Generate a feature vector for each functions pair.
 - 4.2. Perform functions matching using the generated feature vectors to obtain many functions mappings.
 - 4.3. Identify the best functions mapping.
5. Aggregate the match results of each compilation unit and compute the match result of the whole query library.

To match more query libraries, it simply has to repeat the steps from 3 to 5.

3.1.1. LLVM IR Parsing

After the word embedding model has been loaded, the first actual step consists in loading the entire LLVM IR file representing the target binary into memory and map it to a data structure that allows easy retrieving of information from the code, which will be used to create the feature vectors and perform the matching. The LLVM IR allows us to perform parsing that is independent from the architecture for which the target binary is compiled. The data structure into which the target binary is mapped is a composition of other smaller data structures: the smallest ones represent single instructions and global variables; instructions are then structured into basic blocks; one or more basic block compose a function; the set of functions and global variables compose the biggest data structure that represents the entire LLVM IR file. Once a function is mapped into its data structure, we use a sentence composed by the opcodes of its instructions to obtain

an embedding using the word embedding model; this embedding models the semantics of the function.

Once the entire target binary has been parsed and mapped into its data structure, we do the same for the query library: each of its compilation units is mapped using the same data structure described before and an embedding is extracted from each of its functions. So, at the end of the parsing phase, the query library is modelled as a collection of many data structures, one for each of its compilation units, while the target binary is modelled by a single data structure.

3.1.2. Feature Vectors Generation

The feature vectors generation step is performed for each compilation unit of the query library independently. It consists in generating a numerical vector composed by 6 different features extracted from each possible functions pair P . This numerical vector will represents the pair and will be used to perform functions matching. The features that compose each feature vector are the following:

- **Adjusted similarity score:** this feature represents the similarity score between the two functions of the pair; it is the most important feature since the matching approach will use this value to obtain the functions mapping. It is a real number between 0 and 1; it is obtained applying a distance metric to the embeddings of the two functions in the pair and adjusting its value by applying some heuristics based on the values of the other features.
- **Global strings difference:** this feature represents the difference in the global strings found inside the two functions. It is an integer number; for each different global string found it is decreased by one, while it is increased by one for each pair of equal global strings.
- **Called functions difference:** this feature represents the difference in the called functions found inside the instructions of the two functions. It is an integer number; for each different called function found it is decreased by one, while it is increased by one for each pair of equal called functions.
- **Number of blocks difference:** this feature represents the difference between the number of basic blocks of the two functions. It is an integer number lower or equal to zero.
- **Number of arguments difference:** this feature represents the difference between the number of arguments of the two functions. It is an integer number lower or equal to zero.

- **Return type difference:** this feature represents the difference in the return type of the two functions. It is an integer number that can only assume the values 0 (equal return types) or -1 (different return types).

3.1.3. Functions Matching

The functions matching phase has the goal of constructing the best functions mapping $\mathcal{M}$, which is a set of functions pairs. As a starting point for the matching phase our approach selects the best functions pair P' by taking the one with the highest adjusted similarity score. Our approach then constructs many candidate mappings for each compilation unit to find the best mapping $\mathcal{M}$. Each mapping is constructed by selecting the best functions pairs from a subset of the set of all functions pairs; we call these subsets *matching windows*. Each *matching window* must include the best functions pair P' . For each candidate mapping constructed for each *matching window* our approach computes the total similarity score $S = \sum_{i=1}^n s_i$ and it selects as best mapping the one with the highest value for S . At the end of the functions matching phase we have, for each compilation unit of the query library, the best functions mapping $\mathcal{M}$ representing it. These mappings will then be used to compute the aggregate result for the whole query library.

4 | Implementation

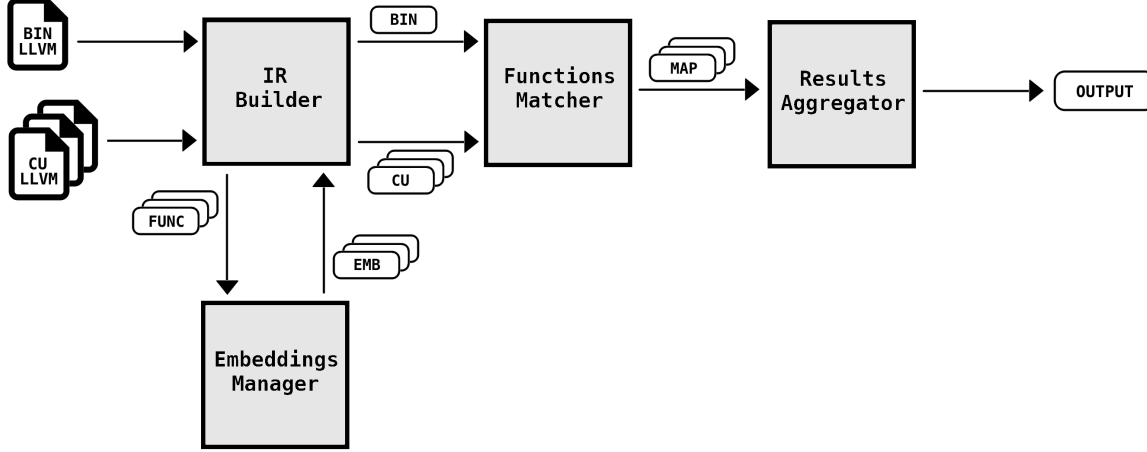
We implement our approach into a Python CLI tool called LIBS, which stands for Libraries Identification in Binaries. LIBS is composed by four different modules: *IR Loader*, *Embeddings Manager*, *Functions Matcher* and *Results Aggregator*. Their relationship, their needed input and their provided output are shown in Figure 4.1.

The initial inputs are the LLVM IR file representing the target binary and the LLVM IR files representing the compilation units of a query library. These files are parsed by the *IR Loader* module and mapped into an internal data structure. During this mapping phase, the *IR Loader* obtains from the *Embeddings Manager* one embedding for each function it is parsing and it stores it inside the data structure itself. The data structures, one representing the whole target binary and the others representing each one a different compilation unit, are then passed to the *Functions Matcher* module which is the one in charge of performing the matching phase to retrieve the mappings between the target binary’s functions and the compilation units’ ones. The matching phase is performed on one compilation unit at a time, by constructing a feature vector for each pair of functions, retrieving many mappings by using these feature vectors, assigning each mapping a score and keeping the mapping with the best score. When a mapping is obtained for each compilation unit, the set of mappings is sent to the *Results Aggregator* module. This module computes the final output by aggregating the scores of the mappings and determines if the query library is present inside the target binary or not.

4.1. *IR Loader*

This module is responsible for parsing LLVM IR files. It takes as input the LLVM IR file representing the target binary and one or more LLVM IR files representing each one a single compilation unit of the query library. Each file is independently parsed using the *llvmlite* [11] Python module and mapped to an internal data structure. This data structure has been designed to model both the binary and the compilation units and to easily retrieve all the information needed for the matching phase. It is a composition of smaller data structures:

Figure 4.1: Architecture of LIBS. The squares represent the modules; the rounded rectangles represent the artifacts produced by the modules; the file icons represent the LLVM IR input files; the arrows represent the input/output relationship between the modules.



- **Global Variable:** this structure represents a global variable of the LLVM IR file; it contains its name and value.
- **Instruction:** this structure represents a single LLVM IR instruction; it contains its opcode, the list of its operands, the list of global variables it uses and the list of functions it calls.
- **Block:** this structure represents a basic block; it contains its name and the list of instructions that belong to it.
- **Function:** this structure represents an entire function of the LLVM IR file; it contains its name, its return type, the list of its parameters, the list of basic blocks that belong to it and the word embedding that represents it.

For each parsed function, the *IR Loader* module constructs from it a sentence composed by the concatenation of all the opcodes of its instructions; this sentence is sent to the *Embeddings Manager* module to retrieve the corresponding word embedding. The output provided by this module is composed by the data structure representing the target binary and the data structures representing the compilation units of the query library which are passed to the *Functions Matcher* module.

4.2. *Embeddings Manager*

This module is the one providing access to a FastText word embedding model, which is used to retrieve a word embedding for each function. To work with the word embedding

model it uses the *fasttext* Python module. It takes as input one sentence composed by the opcodes of all the instructions of a function and provides as output the word embedding for that function.

4.3. *Functions Matcher*

This module is the one in charge of matching the target binary against the query library. To do so, it needs the data structure representing the target binary and the data structures of the compilation units of the query library. It works at the functions level, trying to identify pairs of similar ones, to provide a mapping between the target binary's functions and the ones of the query library. It works by performing the following steps, on one compilation unit at a time:

1. Construct a feature vector for each possible functions pair. This vector is composed by the fields described in Section 3.1.2 and it captures structural, syntactical and semantic properties of the functions pair.
2. Find the best feature vector, i.e., the one that represents the functions pair most likely to be a correct match.
3. Use the best feature vector to extract many different subsets, called *matching windows*, from the set of all the functions pairs.
4. For each *matching window*:
 - 4.1. Construct a cost matrix using the inverse of the similarity score of each functions pair.
 - 4.2. Perform a linear sum assignment (using the *scipy* [12] Python module) on the cost matrix obtaining the best mapping for the *matching window*.
5. Keep the best mapping among the ones obtained above, i.e., the one with the highest total similarity score obtained by the sum of the similarity scores of each functions pair.

At the end of the matching phase, all compilation units have their own best functions mapping; these mappings are then passed to the *Results Aggregator* module.

4.4. *Results Aggregator*

This module takes a set of functions mappings and produces, using them, the final output determining if the query library is present inside the target binary. It starts by computing a match score c for each compilation unit to determine if it is present inside the target binary. To give a positive answer only for compilation units that reach a certain level of

similarity, we introduce a similarity threshold value (sth). If the compilation unit has a match score c greater or equal to this threshold value it is considered positively matched and obtains a "YES" label. The match score computation is shown in (4.1); it is the arithmetic mean of all the similarity scores that are greater or equal to sth .

$$c = \frac{\sum \mathcal{S}}{|\mathcal{S}|}, \quad \mathcal{S} = \{s_i : s_i \geq sth\} \quad (4.1)$$

Considering only the similarity scores above sth has the purpose of not penalizing the match score of the compilation unit in case its mapping contains some functions pair with a low similarity score. However, by only considering the similarity scores above sth , we would consider positively matched every compilation unit with at least one similarity score above the threshold. Thus, we introduce a second threshold, the functions ratio threshold (fth), which establishes the minimum ratio of functions pairs with a similarity score above sth to consider the computed match score valid. This ratio is given by the number of pairs with a similarity score above sth divided by the total number of functions pairs of the compilation unit's mapping. So, all the compilation units that do not reach this ratio value are considered negatively matched and receive a "NO" label.

To better understand the two threshold values sth and fth , look at Figure 4.2 and Figure 4.3. The former shows how, without the similarity threshold, the compilation unit would be wrongly considered negatively matched. The latter shows how, without the functions ratio threshold, the compilation unit would be wrongly considered positively matched.

After the match scores of all the compilation units have been computed, the *Results Aggregator* module computes the match score l of the whole query library to determine if it is present inside the target binary. This computation is shown in (4.2); it is the arithmetic mean of the match scores that are above the value of sth . So it takes into account only the compilation units that have been positively matched.

$$l = \frac{\sum \mathcal{C}}{|\mathcal{C}|}, \quad \mathcal{C} = \{c_i : c_i \geq sth\} \quad (4.2)$$

When the match score l of the query library is computed, two parameters influence the final outcome: they are the minimum number of compilation units (min_cu) and the minimum number of functions inside a compilation unit ($min_functions_cu$). To obtain a "YES" label, a query library must have a number of positively matched compilation units at least equal to the number specified by the min_cu parameter; otherwise, it receives a "NO" label. The second parameter is used to determine if the match scores obtained for

Figure 4.2: Portion of the output of LIBS showing the functions mapping of a compilation unit. The input binary is *curl*, compiled for x86_64 with the O3 optimization level; the query library is *libcurl*; the matched compilation unit is *libcurl_la-llist.o*. The similarity threshold has the value $sth = 0.80$; without it, this compilation unit would have a match score $c = 0.79$ and would be wrongly considered negatively matched.

<i>libcurl_la-llist.o.bc</i>			... [W] [0.99] [1] [5]
<i>Curl_llist_init</i>	--->	<i>Curl_llist_init</i> ...	[0.95]
<i>Curl_llist_insert_next</i>	--->	<i>Curl_llist_insert_next</i> ...	[1.00]
<i>Curl_llist_remove</i>	--->	<i>Curl_llist_remove</i> ...	[1.00]
<i>Curl_llist_destroy</i>	--->	<i>Curl_llist_destroy</i> ...	[0.00]
<i>Curl_llist_count</i>	--->	<i>Curl_llist_count</i> ...	[1.00]

Figure 4.3: Portion of the output of LIBS showing the functions mapping of a compilation unit. The input binary is *curl*, compiled for x86_64 with the O3 optimization level; the query library is *libresolv*; the matched compilation unit is *ns_netint.o*. The functions ratio threshold has the value $fth = 0.80$; without it, this compilation unit would have a match score $c = 1.00$ and would be wrongly considered positively matched.

<i>ns_netint.o.bc</i>			... [0.00] [1] [4]
<i>Curl_read32_le</i>	--->	<i>ns_get16</i> ...	[0.66]
<i>Curl_read16_be</i>	--->	<i>ns_get32</i> ...	[1.00]
<i>Curl_fnmatch</i>	--->	<i>ns_put16</i> ...	[0.39]
<i>Curl_gethostname</i>	--->	<i>ns_put32</i> ...	[0.53]

both the query library and its compilation units can be considered reliable. In fact, for small compilation units composed by only few functions it is easy to find a wrong mapping that has an high total similarity score, since it is easy to find inside the target binary a group of functions that are similar to the few ones of the compilation unit. Instead, it is harder to find a mapping with an high similarity score for a compilation unit with a big number of functions; if such a mapping is found, then it is more reliable than the mapping found for small compilation units. Hence, the *Results Aggregator* module adds a warning flag to all the unreliable match scores obtained. To find which match scores need a warning flag, it uses the *min_functions_cu* parameter: all the compilation units with a "YES" label that have a number of functions below this parameter receive the warning flag. For the query library, if at least one of its positively matched compilation units does not have the warning flag then its match score l is considered reliable; in the other case, it receives the warning flag.

5 | Experimental Validation

To assess the performances of our approach we perform a series of experiments on its implementation, LIBS. Each experiment is meant to test a different ability of the tool. The list of experiments is the following:

- **Experiment 1:** tests the ability of correctly determining which libraries are compiled inside statically linked binaries.
 - **Experiment 1.1:** tests the ability of correctly determining which libraries are compiled inside statically linked binaries when providing the correct libraries versions.
 - **Experiment 1.2:** tests the ability of correctly determining which libraries are compiled inside statically linked binaries when providing the previous minor versions.
 - **Experiment 1.3:** tests the ability of correctly determining which libraries are compiled inside statically linked binaries when providing the previous major versions.
- **Experiment 2:** tests the ability of correctly determining the right library version.
 - **Experiment 2.1:** tests the ability of correctly determining the right library version when providing the correct version and the previous minor version.
 - **Experiment 2.2:** tests the ability of correctly determining the right library version when providing the correct version and the previous major version.
 - **Experiment 2.3:** tests the ability of correctly determining the right *libc* version when providing the correct version and 7 other previous versions.

Each experiment also tests the robustness of our approach in dealing with different compiler versions and different compiler optimizations and is performed for two different software architectures, MIPSel and x86_64. Before conducting our experiments, we need to find the optimal values of the *min_cu* and *min_functions_cu* parameters and the optimal values of the *sth* and *fth* thresholds to obtain the best results for the two software architectures.

5.1. Dataset

To test LIBS we construct a dataset composed by 600 statically linked binaries and 750 static libraries. For the binaries dataset we choose 150 different Linux utilities, which are shown in Table 5.1, and we compile them for the MIPSel architecture and for the x86_64 architecture. The compilation process is conducted using two different Debian-based Docker containers, one for MIPSel and one for x86_64: we download the source code of the binaries and the required static libraries to build them from APT; then we statically compile them using GCC 12.2.0 with two optimization levels, O2 and O3, to obtain a total of 300 MIPSel statically linked binaries and 300 x86_64 ones; we finally use RetDec [13] to lift each binary into the LLVM IR to compose our final dataset of 600 binaries.

For the libraries dataset we get some static libraries from the compilation dependencies of the binaries (True Positives) and some other libraries (True Negatives) among the most common in Linux systems, for a total of 115 different libraries. The full list can be seen in Table 5.2. For each library we download, for each of the two architectures, three different versions (except for the libc, of which we download 8 different versions) already compiled from APT, for a total of 375 MIPSel static libraries and 375 x86_64 ones; we use RetDec to lift each compilation unit of these libraries into the LLVM IR, obtaining our final dataset of 750 libraries. The libraries versions we choose are the following:

- For True Positives, the exact version used to compile the binaries; for True Negatives, the version present in the same APT repository of the True Positives.
- For all, the previous minor version (where available, otherwise another close version).
- For all, the previous major version (where available, otherwise another distant version).

We divide the dataset as follows:

- **Training set:** we use 300 binaries (150 MIPSel and 150 x86_64) to train the word embedding model.
- **Validation set:** we use 152 binaries (76 MIPSel and 76 x86_64) to validate the parameters and thresholds values for LIBS.
- **Testing set:** we use 148 binaries (74 MIPSel and 74 x86_64) to test LIBS with the found optimal parameters and thresholds values.

Finally, the ground truth is composed, for each binary, by the list of libraries that have been used to compile it, together with their exact version.

Table 5.1: List of binaries in the dataset, together with their APT package name.

APT package	Binaries
bash	bash
binutils	addr2line, ar, as, c++filt, elfedit, gprof, ld, nm, objcopy, objdump, ranlib, readelf, size, strings, strip
coreutils	b2sum, base32, base64, basename, basenc, cat, chcon, chgrp, chmod, chown, chroot, cksum, comm, cp, csplit, cut, date, dd, df, dir, dircolors, dirname, du, echo, env, expand, expr, factor, false, fmt, fold, groups, head, hostid, id, join, kill, link, ln, logname, ls, md5sum, mkdir, mkfifo, mknod, mktemp, mv, nice, nl, nohup, nproc, numfmt, od, paste, pathchk, pinky, pr, printenv, printf, ptx, pwd, readlink, realpath, rm, rmdir, runcon, seq, sha1sum, sha224sum, sha256sum, sha384sum, sha512sum, shred, shuf, sleep, sort, split, stat, stty, sum, sync, tac, tail, tee, test, timeout, touch, tr, true, truncate, tsort, tty, uname, unexpand, uniq, unlink, uptime, users, vdir, wc, who, whoami, yes
curl	curl
dash	dash
diffutils	cmp, diff, diff3, sdiff
file	file
findutils	find, locate, xargs
git	git
grep	grep
gzip	gzip
hostname	hostname
less	less
make	make
nano	nano
openssl	openssl
rsync	rsync
sed	sed
socat	socat
tar	tar
util-linux	blkid, fdisk, filan, losetup, mount, procan, sfdisk, umount
vim	vim

5.2. Setup

To run our experiments we use an AWS EC2 *r6a.xlarge* machine with 4 virtual CPUs and 32 GB of RAM with Ubuntu Pro 20.04 LTS. We create a Python virtual environment in which we install all the dependencies of our tool and in which we conduct our tests. Each experiment is handled by a dedicated Python script able to launch LIBS with the correct inputs and parameters, monitor its resources consumption and execution time and save each result to disk.

Table 5.2: List of libraries in the dataset, together with their APT package name.

APT package	Libraries	APT package	Libraries
libacl1-dev	libacl	libmd-dev	libmd
libaio-dev	libaio	libmicrohttpd-dev	libmicrohttpd
libarchive-dev	libarchive	libmpfr-dev	libmpfr
libassuan-dev	libassuan	libncurses-dev	libform, libformw, libmenu, libmenuw, libncurses, libncursesw, libpanel, libpanelw, libtic, libtinfo
libattr1-dev	libattr		
libaudit-dev	libaudit		
libblas-dev	libblas		
libblkid-dev	libblkid	libnhttp2-dev	libnhttp2
libbpf-dev	libbpf	libnhttp3-dev	libnhttp3
libbrotli-dev	libbrotlicommon,	libnsl-dev	libnsl
	libbrotlidec,	libpam0g-dev	libpam_misc, libpam, libpamc
	libbrotlienc		
libbsd-dev	libbsd	libpcre2-dev	libpcre2-8, libpcre2-16, libpcre2-32, libpcre2-posix
libbz2-dev	libbz2	libpng-dev	libpng
libc6-dev	libBrokenLocale,	libpopt-dev	libpopt
	libc_nonshared, libc,	libpq-dev	libpq
	libg, libm, libresolv	libproc2-dev	libproc
libcap-dev	libcap	libpsl-dev	libpsl
libconfig-dev	libconfig	libraptor2-dev	libraptor
libcrypt-dev	libcrypt	librdf0-dev	librdf
libcurl4-openssl-dev	libcurl	libreadline-dev	libhistory, libreadline
libcurl4-gnutls-dev	libcurl-gnutls	librtmp-dev	librtmp
libdbus-1-dev	libdbus	libsasl2-dev	libsasl
libdrm-dev	libdrm_amdgpu,	libseccomp-dev	libseccomp
	libdrm_nouveau,	libsigsegv-dev	libsigsegv
	libdrm_radeon,	libsodium-dev	libsodium
	libdrm	libsqlite3-dev	libsqlite
libelf-dev	libelf	libssh-dev	libssh
libevent-dev	libevent_core,	libssl-dev	libcrypto, libssl
	libevent_extra,	libtasn1-6-dev	libtasn1
	libevent	libtirpc-dev	libtirpc
libexpat1-dev	libexpat	libunistring-dev	libunistring
libffi-dev	libffi	libunwind-dev	libunwind-coredump, libunwind-ptrace, libunwind
libgcrypt20-dev	libgcrypt		
libgdbm-dev	libgdbm		
libgmp-dev	libgmp	libusb-dev	libusb
libgpg-error-dev	libgpg-error	libuv1-dev	libuv
libgpm-dev	libgpm	libwrap0-dev	libwrap
libgsl-dev	libgsl, libgslcblas	libxml2-dev	libxml2
libidn2-dev	libidn	libyaml-dev	libyaml
libjansson-dev	libjansson	libzbd-dev	libzbd
libjson-c-dev	libjson-c	libzstd-dev	libzstd
libkeyutils-dev	libkeyutils	musl-dev	libc
libldap-dev	liblber, libldap	nettle-dev	libhogweed, libnettle
liblinear-dev	liblinear	uuid-dev	libuuid
liblz4-dev	liblz4	zlib1g-dev	libz
liblzma-dev	liblzma		
libmagic-dev	libmagic		
libmbedtls-dev	libmbedcrypto,		
	libmbedtls,		
	libmbedtls509		

5.3. Parameters Validation

As explained in Section 4.4, four parameters influence the final output of the match between a target binary and a query library; these are the minimum number of compilation units (*min_cu*), the minimum number of functions inside a compilation unit (*min_functions_cu*), the similarity threshold (*sth*) and the functions ratio threshold (*fth*). To find

Table 5.3: Optimal configurations of parameters for MIPSel and x86_64.

	MIPSel	x86_64
<i>min_cu</i>	1	1
<i>min_functions_cu</i>	9	11
<i>fth</i>	0.60	0.80
<i>sth</i>	0.80	0.80
Accuracy	1.00	1.00
Precision	0.88	1.00
Recall	0.87	0.88
F1-Score	0.87	0.93

the optimal values for these parameters we run LIBS to match the 76 MIPSel and 76 x86_64 binaries of the validation set against the newest version of the libraries in the dataset, for a total of 115 MIPSel libraries and 115 x86_64 ones for each binary. We use the outcomes of these matches to find the optimal configuration of parameters for MIPSel and for x86_64, by trying many different configurations to see which one produces the best results. As a metric to find the best results we use the F1-Score. We compute it, for each different configuration of parameters, by aggregating the number of True Positives, False Positives, True Negatives and False Negatives of each binary. These numbers are obtained by comparing, for each binary, the list of libraries of its ground truth with the list of positively matched libraries. We consider as positively matched only the libraries with a "YES" label without the warning flag. The ranges we use for the parameters validation are the following:

- *min_cu*: [1, 3]
- *min_functions_cu*: [3, 15]
- *sth*: [0.4, 0.9]
- *fth*: [0.4, 0.9]

So, at the end, we keep the two configurations of parameters that give the maximum F1-Score, one for MIPSel and one for x86_64. These two configurations, reported in Table 5.3, are the ones we propose as default values for LIBS and are the ones we use in our experiments.

5.4. Experiment 1

This experiment is aimed at testing the ability of correctly determining which libraries are compiled inside statically linked binaries when providing:

1. The correct libraries versions.

Table 5.4: Results of Experiment 1.1 for MIPSel and x86_64.

	MIPSel	x86_64
True Positives	106	128
True Negatives	8330	8256
False Positives	16	10
False Negatives	58	36
Accuracy	0.99	0.99
Precision	0.87	0.93
Recall	0.65	0.78
F1-Score	0.74	0.85

2. The previous minor versions.
3. The previous major versions.

5.4.1. Experiment 1.1

For this experiment we use all the 37 binaries in the testing set, each one with both O2 and O3 optimization levels for a total of 74 MIPSel binaries and 74 x86_64 ones. We match these binaries against the newest version of each library in the dataset, for a total of 115 libraries matched against each MIPSel binary and 115 libraries matched against each x86_64 binary; among these newest versions are, as explained in Section 5.1, the correct versions for the libraries that constitute the True Positives.

For each tested binary we obtain a list of 115 answers, one for each library; we use these answers to obtain the number of True Positives, False Positives, True Negatives and False Negatives for each binary. To obtain these numbers we compare, for each binary, the list of libraries of its ground truth with the list of positively matched libraries; we consider as positively matched only the libraries with a "YES" label without the warning flag. Then, by aggregating these numbers, we compute the Precision, Recall and F1-Score metrics for this experiment.

We report these values in Table 5.4. As it can be seen, our approach achieves a good level of Precision in determining if a library is present inside a binary; for the x86_64 architecture this Precision is even higher.

The average time for this experiment is of 7 hours and 15 minutes for MIPSel and 12 hour and 5 minutes for x86_64, while the average memory consumption is of 4 GB for both MIPSel and x86_64 for each single binary.

5.4.2. Experiment 1.2

For this experiment we use 10 binaries from the testing set, each one with both O2 and O3 optimization levels for a total of 40 MIPSel binaries and 40 x86_64 ones. We match these binaries only against libraries that are actually compiled inside them, for a total of 25 different libraries; for each library we provide its previous minor version from the dataset.

For each tested binary we obtain a list of answers, one for each library; we use these answers to determine if our tool is able to produce positive matches even when the tested version is not the correct one. We consider as positively matched all the libraries with a "YES" label.

The results for this experiment are reported in Table 5.5. As it can be seen, our approach achieves good levels of Accuracy in determining if a library is present inside a binary when the provided version is close to the correct one; this time, the results are slightly better for the MIPSel architecture.

The average time for this experiment is of 35 minutes for MIPSel and 1 hour and 15 minutes for x86_64, while the average memory consumption is of 2 GB for MIPSel and 8 GB for x86_64 for each single binary.

5.4.3. Experiment 1.3

For this experiment we use 10 binaries from the testing set, each one with both O2 and O3 optimization levels for a total of 40 MIPSel binaries and 40 x86_64 ones. We match these binaries only against libraries that are actually compiled inside them, for a total of 25 different libraries; for each library we provide its previous major version from the dataset.

For each tested binary we obtain a list of answers, one for each library; we use these answers to determine if our tool is able to produce positive matches even when the tested version is not the correct one. We consider as positively matched all the libraries with a "YES" label.

The results for this experiment are reported in Table 5.6. As it can be seen, our approach achieves slightly worse levels of Accuracy in determining if a library is present inside a binary when the provided version is distant from the correct one.

The average time for this experiment is of 30 minutes for MIPSel and 1 hour and 5 minutes for x86_64, while the average memory consumption is of 2 GB for MIPSel and 7 GB for x86_64 for each single binary.

Table 5.5: Results of Experiment 1.2 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.

	MIPSel		x86_64	
	O2	O3	O2	O3
libacl	✓	✓	✓	✓
libblkid	✓	✓	✓	✓
libbrotlicommon	✓	✓	✗	✗
libbrotlidec	✓	✓	✗	✗
libcap	✓	✓	✓	✓
libcrypto	✓	✓	✓	✓
libcurl	✓	✓	✓	✓
libgmp	✓	✓	✓	✓
libgpm	✗	✓	✗	✗
libidn	✓	✓	✓	✓
libldap	✓	✓	✓	✓
libm	✓	✓	✓	✓
libncursesw	✓	✓	✓	✓
libnhttp2	✓	✓	✓	✓
libpcre2-8	✓	✓	✓	✓
libpopt	✓	✓	✓	✓
libresolv	✓	✓	✓	✓
libsodium	✓	✓	✓	✓
libssl	✓	✓	✓	✓
libtasn1	✓	✓	✓	✓
libtinfo	✓	✓	✓	✓
libuuid	✓	✓	✓	✓
libwrap	✓	✓	✓	✓
libz	✓	✓	✓	✓
libzstd	✓	✓	✓	✓
Correct	24	25	22	22
Wrong	1	0	3	3
Accuracy	0.96	1.00	0.88	0.88

5.5. Experiment 2

This experiment is aimed at testing the ability of correctly determining the right library version when providing:

1. The correct libraries versions and the previous minor versions.
2. The correct libraries versions and the previous major versions.
3. The correct *libc* version and 7 previous versions.

5.5.1. Experiment 2.1

For this experiment we use 10 binaries from the testing set, each one with both O2 and O3 optimization levels for a total of 40 MIPSel binaries and 40 x86_64 ones. We match these binaries only against libraries that are actually compiled inside them, for a total

Table 5.6: Results of Experiment 1.3 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.

	MIPSel		x86_64	
	O2	O3	O2	O3
libacl	✓	✓	✓	✓
libblkid	✓	✓	✓	✓
libbrotlicommon	✗	✗	✗	✗
libbrotlidec	✓	✓	✗	✗
libcap	✗	✗	✗	✗
libcrypto	✓	✓	✓	✓
libcurl	✓	✓	✓	✓
libgmp	✓	✓	✓	✓
libgpm	✗	✓	✗	✗
libidn	✓	✓	✓	✓
libldap	✓	✓	✓	✓
libm	✓	✓	✓	✓
libncursesw	✓	✓	✓	✓
libnghttp2	✓	✓	✓	✓
libpcre2-8	✓	✓	✓	✓
libpopt	✓	✓	✓	✓
libresolv	✓	✓	✓	✓
libsodium	✓	✓	✓	✓
libssl	✓	✓	✓	✓
libtasn1	✓	✓	✓	✓
libtinfo	✓	✓	✓	✓
libuuid	✓	✓	✓	✓
libwrap	✓	✓	✓	✓
libz	✓	✓	✗	✗
libzstd	✗	✗	✗	✗
Correct	21	22	19	19
Wrong	4	3	6	6
Accuracy	0.84	0.88	0.76	0.76

of 25 different libraries; for each library we provide its correct version and its previous minor version from the dataset. For each tested binary we obtain a list of answers, one for each library version; we use these answers to determine if our tool assigns to the correct libraries versions higher scores than the ones of the wrong libraries versions. We determine which version the tool recognizes as the correct one by comparing their answers: a "YES" label without the warning flag wins against one with the warning flag and against a "NO" label; in case of same answers, the higher match score wins; in case of same answers and match scores, the higher number of matched functions wins.

The results for this experiment are reported in Table 5.7, where the mark ✓ means that the tool recognizes the correct library version and the mark ✗ means the opposite; in case of same answers, same match score and same number of matched functions we assign the mark ✗. As it can be seen, our approach doesn't perform very well for the MIPSel architecture, where it struggles to recognize the correct library version among two close

Table 5.7: Results of Experiment 2.1 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.

	MIPSel		x86_64	
	O2	O3	O2	O3
libacl	✗	✗	✓	✓
libblkid	✓	✓	✓	✓
libbrotlicommon	✓	✓	✓	✓
libbrotlidec	✓	✓	✓	✓
libcap	✓	✓	✓	✓
libcrypto	✗	✗	✓	✓
libcurl	✓	✓	✓	✓
libgmp	✓	✓	✓	✓
libgpm	✗	✓	✓	✓
libidn	✗	✗	✓	✓
libldap	✗	✗	✗	✗
libm	✓	✓	✓	✗
libncursesw	✓	✓	✓	✓
libnhttp2	✗	✗	✗	✗
libpcre2-8	✗	✗	✓	✓
libpopt	✓	✓	✓	✓
libresolv	✗	✗	✗	✗
libsodium	✗	✗	✓	✓
libssl	✓	✓	✓	✓
libtasn1	✗	✗	✗	✗
libtinfo	✓	✓	✓	✓
libuuid	✗	✓	✓	✓
libwrap	✓	✓	✓	✓
libz	✗	✓	✗	✗
libzstd	✓	✓	✓	✓
Correct	13	16	20	19
Wrong	12	9	5	6
Accuracy	0.52	0.64	0.80	0.76

ones; instead, for the x86_64 architecture it achieves a good level of Accuracy.

The average time for this experiment is of 35 minutes for MIPSel and 1 hour and 15 minutes for x86_64, while the average memory consumption is of 2 GB for MIPSel and 8 GB for x86_64 for each single binary.

5.5.2. Experiment 2.2

For this experiment we use 10 binaries from the testing set, each one with both O2 and O3 optimization levels for a total of 40 MIPSel binaries and 40 x86_64 ones. We match these binaries only against libraries that are actually compiled inside them, for a total of 25 different libraries; for each library we provide its correct version and its previous major version from the dataset.

For each tested binary we obtain a list of answers, one for each library version; we use these

answers to determine if our tool assigns to the correct libraries versions higher scores than the ones of the wrong libraries versions. We determine which version the tool recognizes as the correct one by comparing their answers: a "YES" label without the warning flag wins against one with the warning flag and against a "NO" label; in case of same answers, the higher match score wins; in case of same answers and match scores, the higher number of functions matched wins.

The results for this experiment are reported in Table 5.8, where the mark ✓ means that the tool recognizes the correct library version and the mark ✗ means the opposite; in case of same answers, same match score and same number of matched functions we assign the mark ✗. As it can be seen, our approach achieves good levels of Accuracy in recognizing the correct library version among two distant ones; as before, the results are better for the x86_64 architecture.

The average time for this experiment is of 30 minutes for MIPSel and 1 hour and 5 minutes for x86_64, while the average memory consumption is of 2 GB for MIPSel and 7 GB for x86_64 for each single binary.

5.5.3. Experiment 2.3

For this experiment we use 10 binaries from the testing set, each one with both O2 and O3 optimization levels for a total of 40 MIPSel binaries and 40 x86_64 ones. We match these binaries only against the *libc*, which is actually compiled inside them; we provide its exact version (2.36-9+deb12u3) together with 7 older versions (2.35-4, 2.33-8, 2.31-17, 2.29-10, 2.27-8, 2.25-6, 2.23-5), for a total of 8 different *libc* versions.

For each tested binary we obtain a list of answers, one for each *libc* version; we use these answers to determine if our tool assigns to the correct *libc* version higher scores than the ones of the wrong versions. We determine which version the tool recognizes as the correct one by comparing their answers: a "YES" label without the warning flag wins against one with the warning flag and against a "NO" label; in case of same answers, the higher match score wins; in case of same answers and match scores, the higher number of functions matched wins.

The results for this experiment are reported in Table 5.9, where the mark ✓ means that the tool recognizes the correct *libc* version and the mark ✗ means the opposite; in case of same answers, same match score and same number of matched functions we assign the mark ✗. As it can be seen, our approach is able to determine the correct *libc* version for all the tested binaries, for both MIPSel and x86_64.

The average time for this experiment is of 1 hour and 45 minutes for MIPSel and 2 hours

Table 5.8: Results of Experiment 2.2 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.

	MIPSel		x86_64	
	O2	O3	O2	O3
libacl	✗	✗	✓	✓
libblkid	✓	✓	✓	✓
libbrotlicommon	✓	✓	✓	✓
libbrotlidec	✓	✓	✓	✓
libcap	✓	✓	✓	✓
libcrypto	✗	✗	✓	✓
libcurl	✓	✓	✓	✓
libgmp	✓	✓	✓	✓
libgpm	✓	✗	✓	✓
libidn	✗	✗	✓	✓
libldap	✗	✗	✗	✗
libm	✓	✗	✗	✗
libncursesw	✓	✓	✓	✓
libnghttp2	✓	✓	✓	✓
libpcre2-8	✓	✓	✓	✓
libpopt	✓	✓	✓	✓
libresolv	✗	✗	✗	✗
libsodium	✓	✓	✓	✓
libssl	✓	✓	✓	✓
libtasn1	✗	✗	✗	✗
libtinfo	✓	✓	✓	✓
libuuid	✓	✓	✓	✓
libwrap	✗	✓	✓	✓
libz	✓	✓	✓	✓
libzstd	✓	✓	✓	✓
Correct	18	17	21	21
Wrong	7	8	4	4
Accuracy	0.72	0.68	0.84	0.84

and 50 minutes for x86_64, while the average memory consumption is of 2 GB for MIPSel and 2.5 GB for x86_64 for each single binary.

Table 5.9: Results of Experiment 2.3 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.

	MIPSel		x86_64	
	O2	O3	O2	O3
cat	✓	✓	✓	✓
cmp	✓	✓	✓	✓
elfedit	✓	✓	✓	✓
ld	✓	✓	✓	✓
pwd	✓	✓	✓	✓
sort	✓	✓	✓	✓
touch	✓	✓	✓	✓
uptime	✓	✓	✓	✓
whoami	✓	✓	✓	✓
xargs	✓	✓	✓	✓
Correct	10	10	10	10
Wrong	0	0	0	0
Accuracy	1.00	1.00	1.00	1.00

5.6. Use Cases Flexibility

As explained in Section 5.3, for LIBS we provide two default configurations of parameters, one for MIPSel and one for x86_64; these are the ones that have the highest F1-Score, so the ones with the optimal trade-off between the Precision and the Recall metrics. However, in some situations we would like to maximize one metric at the expense of the other: when we want to obtain few False Positives we need to maximize the Precision metric; when instead we want to obtain few False Negatives we need to maximize the Recall metric.

In Table 5.10 we report the maximum Recall obtained for some high Precision values and in Table 5.11 we report the maximum Precision obtained for some high Recall values, together with the configurations of parameters to obtain them. These configurations can be used to tweak LIBS to adapt it to each specific situation.

Table 5.10: Maximum Recall obtained for specific Precision values, together with the configurations of parameters (min_cu , $min_functions_cu$, sth , fth) to obtain them.

MIPSel			x86_64		
Precision	Recall	Parameters	Precision	Recall	Parameters
1.00	0.76	(1, 7, 0.90, 0.60)	1.00	0.88	(1, 11, 0.80, 0.80)
0.95	0.76	(1, 9, 0.70, 0.80)	0.95	0.89	(3, 5, 0.90, 0.60)
0.90	0.76	(3, 10, 0.70, 0.80)	0.90	0.91	(2, 11, 0.50, 0.90)
0.85	0.82	(1, 6, 0.70, 0.80)	0.85	0.91	(1, 9, 0.50, 0.90)
0.80	0.84	(1, 9, 0.70, 0.70)	0.80	0.90	(1, 6, 0.70, 0.70)
0.75	0.59	(3, 15, 0.50, 0.90)	0.75	0.88	(2, 12, 0.50, 0.80)
0.70	0.76	(1, 11, 0.80, 0.60)	0.70	0.88	(3, 10, 0.90, 0.40)

Table 5.11: Maximum Precision obtained for specific Recall values, together with the configurations of parameters (min_cu , $min_functions_cu$, sth , fth) to obtain them.

MIPSel			x86_64		
Recall	Precision	Parameters	Recall	Precision	Parameters
1.00	0.10	(1, 3, 0.80, 0.60)	1.00	0.28	(1, 5, 0.60, 0.80)
0.95	0.24	(1, 9, 0.80, 0.50)	0.95	0.90	(3, 11, 0.50, 0.90)
0.90	0.26	(1, 3, 0.70, 0.80)	0.90	0.80	(1, 6, 0.70, 0.70)
0.85	0.33	(3, 10, 0.90, 0.40)	0.85	0.98	(3, 5, 0.90, 0.80)
0.80	0.94	(1, 8, 0.70, 0.80)	0.80	1.00	(1, 12, 0.90, 0.80)
0.75	0.93	(1, 9, 0.60, 0.90)	0.75	1.00	(2, 11, 0.90, 0.90)
0.70	1.00	(1, 12, 0.80, 0.80)	0.70	1.00	(3, 11, 0.90, 0.80)

6 | Limitations and Future Works

The approach we propose demonstrates to achieve good results, even when dealing with different compiler settings and different libraries versions. However, experimental validation shows some limitations in recognizing the correct libraries versions when comparing between two close ones. This can be seen as an inherent limitation of our approach since we base the libraries identification on the match of each compilation unit independently. This limitation derives from the fact that not all compilation units of a library are normally included inside the binary; thus, if between two close library versions the changes regard only few compilation units that are not imported into the binary, it is impossible for our approach to distinguish between them, since the actual compilation units inside the binary are identical. This problem, as seen in experimental validation, tends to vanish the more distance there is between the library versions, because it is more likely that some changes occurred in the compilation units that are imported into the binary. Another limitation that emerges from experimental validation is the considerable amount of time and resources needed when dealing with large binaries or large libraries. This can be attributed to the many, non-parallelized linear sum assignments that are computed for each compilation unit; in fact, the bigger the number of entries in the cost matrix (due to big compilation units), the longer it needs to produce a mapping.

To address this last problem, two possible future improvements can be considered: the first would be to parallelize the computation of these linear sum assignments; the second would be to use a linear sum assignment implementation that makes use of a GPU instead of a CPU, which is the case for the *scipy*'s implementation used in our tool. Other future works could involve testing the approach on other architectures besides MIPSel and x86_64 to find the best configuration of parameters for those architectures. An interesting future work could be to adapt our approach to patch searching, which involves finding if two binaries, which theoretically are compiled from the same source code, are instead compiled from slightly different code due to the presence of a patch. Our approach could be used in this scenario as follows:

- The first input would be the LLVM IR of the target binary.
- The second input, instead of many LLVM IR files of libraries, would be the LLVM IR file of the binary compiled from the original source code.
- The match phase would produce one single mapping containing many functions pairs with a similarity score of 1.00 since they represent perfectly equal functions and zero or few functions pairs with a similarity score lower than 1.00.
- If there are no functions pairs in the mapping with a similarity score lower than 1.00 it means that the target binary has not been patched; instead, if there are functions pairs with a similarity score lower than 1.00 it means that these functions could possibly contain a patch and must be manually inspected. The output should consist of only functions pairs with a similarity score lower than 1.00.

To extend our approach to this new scenario, some tweaks in the internal heuristics and in the configuration of parameters would be probably needed; so a possible future work could be to test the approach for patch searching to find the correct parameters values that allow to perform this task.

7 | Conclusions

This thesis proposes an approach that performs library functions identification to identify which libraries have been compiled inside a statically linked binary and to recognize their correct version. This approach could be useful in some binary code analysis areas such as bug searching and reverse engineering.

Many challenges hinder the achievement of this goal; the biggest issue is represented by the differences in compiled code introduced by different compilation settings and different software architectures. Another important issue is the multiplicity of libraries versions, that makes it hard to identify the correct one. To overcome these challenges we propose an approach that takes into account different structural, syntactical and semantic features of the decompiled code; this heterogeneity allows our approach to be robust against those code variations. The approach we propose is based on a word embedding model to extract the semantics from each function and it works on top of an intermediate representation of the code, the LLVM IR, to be architecture-independent.

Our approach is implemented into a Python tool called LIBS and is tested on two different software architectures, MIPSel and x86_64, to assess its performances. Experimental validation demonstrates that LIBS can achieve good results in identifying which libraries have been compiled inside a statically linked binary, with a Precision of 87% for MIPSel and 93% for x86_64. Moreover, experimental results show that our approach is able to identify the libraries even between different versions and to correctly provide their exact version with an Accuracy of 72% for MIPSel and 84% for x86_64.

Experimental validation also highlights some limitations of our approach; the most notable ones are the difficulty in recognizing the correct library version between too close versions and the time and resources needed when dealing with large binaries or libraries. The first limitation is a consequence of the matching phase of our approach, which works on each compilation unit independently. For the second limitation, a possible future improvement could be to parallelize the computation of the many linear sum assignments performed during the matching phase.

Bibliography

- [1] Hex-Rays. IDA Pro (available at <https://hex-rays.com/ida-pro>).
- [2] Hex-Rays. FLIRT (available at <https://hex-rays.com/products/ida/tech/flirt>).
- [3] Jing Qiu, Xiaohong Su, and Peijun Ma. Library Functions Identification in Binary Code by using Graph Isomorphism Testings. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 2015.
- [4] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 2017.
- [5] Zynamics. BinDiff (available at <https://www.zynamics.com/bindiff.html>).
- [6] Martial Bourquin, Andy King, and Edward Robbins. BinSlayer: Accurate Comparison of Binary Executables. In *Proceedings of the 2nd ACM SIGPLAN Program Protection and Reverse Engineering Workshop*, 2013.
- [7] Joxean Koret. Diaphora (available at <http://diaphora.re>).
- [8] Yue Duan, Xuezixiang Li, Jinghan Wang, and Heng Yin. DeepBinDiff: Learning Program-wide Code Representations for Binary Diffing. In *Proceedings of the Network and Distributed System Security Symposium*, 2020.
- [9] Xiaojun Xu, Chang Liu, Qian Feng, Heng Yin, Le Song, and Dawn Song. Neural Network-based Graph Embedding for Cross-Platform Binary Code Similarity Detection. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017.
- [10] Bingchang Liu, Wei Huo, Chao Zhang, Wenchao Li, Feng Li, Aihua Piao, and Wei Zou. α Diff: Cross-Version Binary Code Similarity Detection with DNN. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018.
- [11] Numba. llvmlite (available at <https://github.com/numba/llvmlite>).

- [12] SciPy. SciPy (available at <https://github.com/scipy/scipy>).
- [13] Avast. RetDec (available at <https://github.com/avast/retdec>).

List of Figures

- 4.1 Architecture of LIBS. The squares represent the modules; the rounded rectangles represent the artifacts produced by the modules; the file icons represent the LLVM IR input files; the arrows represent the input/output relationship between the modules. 12
- 4.2 Portion of the output of LIBS showing the functions mapping of a compilation unit. The input binary is *curl*, compiled for x86_64 with the O3 optimization level; the query library is *libcurl*; the matched compilation unit is *libcurl_la-llist.o*. The similarity threshold has the value $sth = 0.80$; without it, this compilation unit would have a match score $c = 0.79$ and would be wrongly considered negatively matched. 15
- 4.3 Portion of the output of LIBS showing the functions mapping of a compilation unit. The input binary is *curl*, compiled for x86_64 with the O3 optimization level; the query library is *libresolv*; the matched compilation unit is *ns_netint.o*. The functions ratio threshold has the value $fth = 0.80$; without it, this compilation unit would have a match score $c = 1.00$ and would be wrongly considered positively matched. 15

List of Tables

5.1	List of binaries in the dataset, together with their APT package name. . .	19
5.2	List of libraries in the dataset, together with their APT package name. . .	20
5.3	Optimal configurations of parameters for MIPSel and x86_64.	21
5.4	Results of Experiment 1.1 for MIPSel and x86_64.	22
5.5	Results of Experiment 1.2 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.	24
5.6	Results of Experiment 1.3 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.	25
5.7	Results of Experiment 2.1 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.	26
5.8	Results of Experiment 2.2 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.	28
5.9	Results of Experiment 2.3 for MIPSel and x86_64. The ✓ symbol represents a correct match and the ✗ symbol represents a wrong match.	29
5.10	Maximum Recall obtained for specific Precision values, together with the configurations of parameters (<i>min_cu</i> , <i>min_functions_cu</i> , <i>sth</i> , <i>fth</i>) to obtain them.	30
5.11	Maximum Precision obtained for specific Recall values, together with the configurations of parameters (<i>min_cu</i> , <i>min_functions_cu</i> , <i>sth</i> , <i>fth</i>) to obtain them.	30

Ringraziamenti

Vorrei ringraziare il Prof. Mario Polino e Lorenzo Binosi per avermi dato l'opportunità di svolgere questo lavoro e per avermi guidato con costanza e disponibilità.

Vorrei inoltre ringraziare la mia famiglia, soprattutto mia mamma e mia sorella, per avermi permesso di intraprendere questo percorso universitario, sostenendomi sotto ogni aspetto e supportando ogni mia scelta. Grazie di cuore, vi voglio bene.

Un ringraziamento molto speciale va a Valentina, la mia dolce metà, per l'enorme pazienza e la comprensione dimostrate durante tutto il mio percorso di studi e per il sostegno che mi dà sempre.

Infine, ringrazio tutti i miei cari amici per essermi stati vicini in questi anni.

